

Profiling-Driven Optimization of a GPU Inference Pipeline for Wafer Defect Classification

Sumin Sim
Technical Report
September 2026

Abstract

This report presents a profiling-driven performance analysis and optimization of a C++/CUDA inference pipeline for semiconductor wafer defect classification. A lightweight CNN trained on the WM-811K dataset was deployed using ONNX Runtime with the CUDA Execution Provider. Rather than assuming GPU computation was the primary performance bottleneck, the system was decomposed and measured across CPU preprocessing, host-to-device transfer, GPU inference, and end-to-end execution. Profiling revealed that disk image loading and decoding dominated the original pipeline, accounting for approximately 95.4% of end-to-end latency at batch size 64. A RAM-cached controlled experiment was then used to isolate GPU execution from disk I/O. Under the isolated workload, FP32 and FP16 inference were compared across multiple batch sizes. Despite FP16 Tensor Core execution, FP16 provided no throughput advantage for this lightweight network. Based on the measured results, FP32 was retained as the final inference configuration. The results demonstrate the importance of measurement-driven optimization and workload-specific performance analysis in GPU inference systems.

1. Introduction

Machine learning inference performance is determined not only by the computational cost of a model, but also by the efficiency of the surrounding systems pipeline. Data loading, preprocessing, memory transfers, runtime execution, and hardware utilization can each become performance bottlenecks, making theoretical GPU capability alone an insufficient predictor of end-to-end performance. For this reason, effective inference optimization requires empirical measurement of the complete execution pipeline rather than optimization based solely on assumptions about model computation.

This report investigates these performance characteristics through a C++/CUDA inference pipeline for semiconductor wafer defect classification. A lightweight convolutional neural network (CNN), trained on the WM-811K wafer map dataset and exported to ONNX, was deployed using the ONNX Runtime C++ API with the CUDA Execution Provider. The system was evaluated across multiple batch sizes to examine end-to-end latency, CPU preprocessing cost, host-to-device memory transfer, GPU inference latency, and throughput.

The primary objective of this work was to identify the actual performance bottleneck of the inference pipeline and determine which optimizations produced measurable improvements. Rather than assuming that GPU computation was the dominant cost, the pipeline was decomposed into individual execution stages and analyzed through controlled experiments. After the initial measurements revealed a significant CPU-side input bottleneck, a RAM-cached benchmark was introduced to isolate GPU execution from disk loading and image decoding.

The isolated pipeline was then used to perform a controlled comparison between FP32 and FP16 inference. Although FP16 is commonly associated with improved GPU inference performance, the experiments showed that it did not provide a throughput advantage for this lightweight CNN. The resulting analysis demonstrates a measurement-driven optimization process in which profiling, bottleneck isolation, and controlled benchmarking are used to select an inference configuration based on observed workload behavior rather than theoretical hardware capability.

2. System and Experimental Setup

2.1 Model and Inference Pipeline

The target workload is a lightweight convolutional neural network (CNN) for semiconductor wafer defect classification using the WM-811K wafer map dataset. The model accepts a 24×24 single-channel wafer map as input and predicts one of eight defect classes. The network was originally developed and trained in PyTorch and subsequently exported to ONNX for deployment.

For standalone inference, the model was integrated into a C++17 application using the ONNX Runtime C++ API. OpenCV was used for image loading and preprocessing, while GPU execution was enabled through the ONNX Runtime CUDA Execution Provider. Both

FP32 and FP16 ONNX models were evaluated to investigate the effect of numerical precision on inference performance.

The end-to-end inference pipeline consisted of four primary stages:

Input Image → OpenCV Preprocessing → Host-to-Device Transfer → GPU Inference

Separating these stages allowed the contribution of CPU-side processing, GPU memory transfer, and GPU execution to be measured independently.

2.2 Hardware and Software Environment

All performance experiments were conducted on the same local system to maintain a consistent execution environment. The software and hardware configuration used for benchmarking is summarized in Table 1.

Table 1. Experimental environment

Component	Configuration
Operating System	Windows
GPU	NVIDIA GeForce GTX 1660 (6 GB VRAM)
NVIDIA Driver	561.17
CUDA Toolkit	12.6
cuDNN	9.8
ONNX Runtime GPU	1.26.0
OpenCV	4.14.0
C++ Standard	C++17
CMake	4.4.2

2.3 Benchmark Methodology

Inference performance was evaluated at batch sizes of 1, 8, 16, 32, and 64. The benchmarks measured CPU preprocessing latency, host-to-device (H2D) transfer latency, pure GPU inference latency, total end-to-end (E2E) latency, and throughput in frames per second (FPS).

The experiments were conducted progressively rather than treating the inference pipeline as a single black box. An initial batch-scaling benchmark established baseline performance. The pipeline was then instrumented to measure individual latency components using ONNX Runtime I/O Binding. After the dominant CPU-side input bottleneck was identified, all 3,828 test images were preloaded into system memory to create a RAM-cached benchmark that isolated inference execution from disk I/O and image decoding.

Finally, the RAM-cached configuration was used as the controlled environment for comparing FP32 and FP16 execution. This experimental design ensured that the precision comparison primarily reflected differences in inference execution rather than variations in disk access or image-loading overhead.

3. Performance Analysis and Bottleneck Identification

3.1 Baseline Batch Scaling

The first experiment established baseline end-to-end performance across batch sizes of 1, 8, 16, 32, and 64. Increasing the batch size improved throughput by allowing more samples to be processed during each inference call. However, the initial results also showed that overall throughput was constrained by operations outside the GPU inference stage.

These observations motivated a more detailed decomposition of end-to-end latency. Rather than optimizing GPU execution immediately, the pipeline was instrumented to determine how much time was spent in preprocessing, host-to-device transfer, and GPU inference.

3.2 Latency Decomposition

ONNX Runtime I/O Binding was introduced to support more controlled measurement of the execution pipeline. End-to-end latency was decomposed into CPU preprocessing, host-to-device (H2D) transfer, and GPU inference.

At batch size 64, the measured latency breakdown was:

Table 2. End-to-end latency breakdown at batch size 64

Stage	Latency	Share
CPU Preprocessing (Disk Read + OpenCV Decode)	10.717 ms	~95.4%
H2D Transfer	0.037 ms	~0.3%
GPU Inference	0.482 ms	~4.3%
Total E2E	11.236 ms	100.0%

The results revealed that approximately 95.4% of total end-to-end latency originated from the CPU-side input stage. In comparison, H2D transfer represented only a small fraction of execution time, while GPU inference completed in less than half a millisecond.

This result changed the direction of the optimization process. The GPU was completing inference substantially faster than the input pipeline could supply tensors. Therefore, additional GPU-side optimization alone was unlikely to produce a meaningful improvement in end-to-end performance.

```

=== Latency Breakdown Analysis (FP32) ===
Batch | Preprocess (A) | H2D Transfer | GPU Infer (B) | Total E2E (C) | Prep Share
-----|-----|-----|-----|-----|-----
Batch 1 | 0.202 ms | 0.006 ms | 0.154 ms | 0.361 ms | 55.836 %
Batch 8 | 1.407 ms | 0.016 ms | 0.185 ms | 1.608 ms | 87.479 %
Batch 16 | 2.724 ms | 0.023 ms | 0.248 ms | 2.994 ms | 90.968 %
Batch 32 | 5.310 ms | 0.025 ms | 0.353 ms | 5.689 ms | 93.339 %
Batch 64 | 10.717 ms | 0.037 ms | 0.482 ms | 11.236 ms | 95.379 %

```

Figure 1. Latency breakdown across batch sizes, showing the dominance of CPU-side image loading and preprocessing.

4. Controlled Bottleneck Isolation

4.1 RAM-Cached Experiment

The preprocessing measurement in the original pipeline included disk image loading and OpenCV image decoding. A second experiment was therefore designed to determine whether the observed CPU-side bottleneck was caused primarily by disk I/O or by the tensor preprocessing operations themselves.

All 3,828 test images were loaded into system memory before benchmark timing began. The resulting RAM-cached pipeline removed repeated disk reads from the timed execution path:

Original Pipeline:

Disk → Image Decode → CPU Preprocessing → H2D → GPU Inference

RAM-Cached Pipeline:

RAM Cache → CPU Preprocessing → H2D → GPU Inference

This modification served as a controlled experiment rather than simply a performance optimization. By removing disk access while retaining the remaining inference stages, the contribution of disk loading and decoding could be isolated.

4.2 RAM-Cached Results

At batch size 64, removing repeated disk access substantially reduced end-to-end latency.

Table 3. Original and RAM-cached pipeline comparison at batch size 64

Metric	Original Pipeline	RAM-Cached Pipeline
Preprocessing Latency	10.717 ms	0.028 ms
GPU Inference Latency	0.482 ms	0.321 ms
Total E2E Latency	11.236 ms	0.359 ms

The preprocessing stage decreased by more than two orders of magnitude after the images were cached in memory. Total end-to-end latency decreased from 11.236 ms to 0.359 ms, representing approximately a 31× reduction in measured E2E latency.

The experiment therefore provided direct evidence that disk image loading and decoding, rather than GPU execution or H2D transfer, dominated the original pipeline.

The RAM-cached benchmark should not be interpreted as normal file-based application throughput. Instead, it represents an intentionally isolated execution environment designed to expose the inference capacity of the underlying pipeline without repeated storage access.

Figure 2. RAM-cached benchmark after removing repeated disk I/O from the timed execution path.

```

=== RAM-Cached Latency Breakdown Analysis (FP32) ===
Batch | Preprocess (A) | H2D Transfer | GPU Infer (B) | Total E2E (C) | Throughput (FPS)
-----|-----|-----|-----|-----|-----
Batch 1 | 0.002 ms | 0.003 ms | 0.139 ms | 0.145 ms | 6915.217 FPS
Batch 8 | 0.005 ms | 0.009 ms | 0.159 ms | 0.173 ms | 46158.799 FPS
Batch 16 | 0.011 ms | 0.010 ms | 0.208 ms | 0.229 ms | 69642.925 FPS
Batch 32 | 0.020 ms | 0.008 ms | 0.257 ms | 0.284 ms | 112131.793 FPS
Batch 64 | 0.038 ms | 0.011 ms | 0.343 ms | 0.392 ms | 162548.461 FPS

```

5. FP32 vs. FP16 Precision Evaluation

5.1 Experimental Motivation

After disk I/O was isolated, the remaining pipeline provided a more appropriate environment for comparing GPU execution modes. FP16 inference is commonly used to improve GPU throughput by reducing numerical precision and enabling specialized hardware execution. However, whether FP16 improves performance depends on the characteristics of the target workload.

To evaluate this experimentally, FP32 and FP16 ONNX models were executed under the same RAM-cached benchmark conditions across batch sizes of 1, 8, 16, 32, and 64.

5.2 Performance Results

Contrary to the initial expectation, FP16 did not outperform FP32 for the target CNN.

Table 4. FP32 and FP16 inference performance

Batch	FP32 Infer	FP16 Infer	FP32 FPS	FP16 FPS
1	0.139 ms	0.180 ms	6,944	5,390
8	0.165 ms	0.225 ms	44,413	33,539
16	0.215 ms	0.273 ms	67,744	54,699
32	0.259 ms	0.384 ms	112,033	77,823
64	0.349 ms	0.543 ms	161,305	107,882

Across the tested batch sizes, FP16 consistently provided lower performance than FP32. At batch size 64, FP32 achieved approximately 161,305 FPS in the isolated benchmark compared with 107,882 FPS for FP16.

This result demonstrates that reduced numerical precision does not automatically translate into improved inference throughput.

Figure 3. Controlled RAM-cached comparison of FP32 and FP16 inference across batch sizes 1–64.

RAM-Cached Fair FP32 vs. FP16 Benchmark (Latency, VRAM & Accuracy)							
[Batch Size: 1]							
Precision	Prep (ms)	H2D (ms)	Pure GPU Infer (ms)	Total E2E (ms)	Isolated FPS	VRAM Allocated (MB)	
FP32	0.002	0.003	0.136	0.141	7091.489	1031.688	
FP16	0.002	0.003	0.177	0.182	5484.786	1031.688	
-> Pure GPU Infer Speedup (FP16/FP32) : 0.77x							
-> Total E2E Speedup (FP16/FP32) : 0.77x							
-> Accuracy Output Divergence : Max Absolute Error = 6.238937e-03 Mean Absolute Error = 1.788363e-03							
[Batch Size: 8]							
Precision	Prep (ms)	H2D (ms)	Pure GPU Infer (ms)	Total E2E (ms)	Isolated FPS	VRAM Allocated (MB)	
FP32	0.005	0.009	0.165	0.180	44410.053	1031.688	
FP16	0.005	0.009	0.223	0.236	33861.902	1031.688	
-> Pure GPU Infer Speedup (FP16/FP32) : 0.74x							
-> Total E2E Speedup (FP16/FP32) : 0.76x							
-> Accuracy Output Divergence : Max Absolute Error = 7.411957e-03 Mean Absolute Error = 2.098401e-03							
[Batch Size: 16]							
Precision	Prep (ms)	H2D (ms)	Pure GPU Infer (ms)	Total E2E (ms)	Isolated FPS	VRAM Allocated (MB)	
FP32	0.008	0.009	0.190	0.207	76936.682	1033.688	
FP16	0.008	0.009	0.261	0.278	57298.870	1031.688	
-> Pure GPU Infer Speedup (FP16/FP32) : 0.73x							
-> Total E2E Speedup (FP16/FP32) : 0.74x							
-> Accuracy Output Divergence : Max Absolute Error = 7.411957e-03 Mean Absolute Error = 2.098401e-03							
[Batch Size: 32]							
Precision	Prep (ms)	H2D (ms)	Pure GPU Infer (ms)	Total E2E (ms)	Isolated FPS	VRAM Allocated (MB)	
FP32	0.018	0.008	0.243	0.268	118819.626	1033.688	
FP16	0.015	0.007	0.367	0.389	81961.948	1033.688	
-> Pure GPU Infer Speedup (FP16/FP32) : 0.66x							
-> Total E2E Speedup (FP16/FP32) : 0.69x							
-> Accuracy Output Divergence : Max Absolute Error = 9.384155e-03 Mean Absolute Error = 1.719991e-03							
[Batch Size: 64]							
Precision	Prep (ms)	H2D (ms)	Pure GPU Infer (ms)	Total E2E (ms)	Isolated FPS	VRAM Allocated (MB)	
FP32	0.027	0.010	0.315	0.351	181797.458	1039.688	
FP16	0.026	0.010	0.501	0.537	118812.619	1033.688	
-> Pure GPU Infer Speedup (FP16/FP32) : 0.63x							
-> Total E2E Speedup (FP16/FP32) : 0.65x							
-> Accuracy Output Divergence : Max Absolute Error = 1.172829e-02 Mean Absolute Error = 1.890511e-03							

6. Analysis of FP16 Performance Inversion

The FP16 performance result was counterintuitive because profiling confirmed execution of an FP16 Tensor Core-capable kernel: sm75_xmma_fprop_implicit_gemm

This observation indicates that the FP16 performance degradation was not simply the result of failing to execute an FP16-capable GPU kernel. Instead, the measured behavior is consistent with the characteristics of the workload and runtime overhead.

6.1 Small Computational Workload

The CNN operates on 24×24 single-channel inputs and has a relatively small computational workload. As a result, individual inference operations may not provide enough work to fully exploit the higher arithmetic throughput available from reduced-precision execution.

For workloads of this size, fixed execution costs can represent a significant fraction of total latency. Consequently, increasing theoretical arithmetic throughput does not necessarily reduce observed inference latency.

6.2 Runtime and Launch Overhead

GPU inference involves more than mathematical computation. Kernel launches, runtime dispatch, synchronization, and other fixed execution costs contribute to total latency.

When model computation is relatively small, these costs become proportionally more significant. The potential compute advantage of FP16 can therefore be insufficient to compensate for the surrounding execution overhead.

The benchmark results illustrate an important systems principle: hardware capabilities must be evaluated in the context of the actual workload rather than treated as guarantees of application-level performance.

7. Numerical Precision and Model Quality

Performance alone is not sufficient for selecting an inference precision. The effect of FP16 conversion on model outputs and classification quality was therefore evaluated against the FP32 model.

At batch size 64, the maximum absolute difference between FP32 and FP16 output logits was approximately 1.172×10^{-2} , while the mean absolute difference was approximately 1.89×10^{-3} .

Classification performance remained nearly unchanged:

Table 5. FP32 and FP16 model quality

Metric	FP32	FP16	Absolute Delta
Accuracy	85.08%	85.06%	-0.02%
Weighted F1-Score	0.8544	0.8542	-0.0002

These results indicate that FP16 preserved essentially the same classification quality as FP32 for this model. However, because FP16 did not provide a throughput advantage, the small reduction in numerical precision offered no practical performance benefit for the target workload.

8. Engineering Decision and Discussion

The experiments demonstrate why inference optimization should be approached as a systems problem rather than solely as a model or GPU optimization problem.

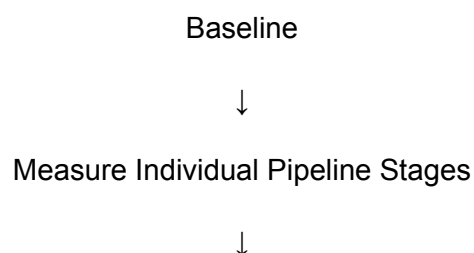
Detailed latency decomposition revealed that approximately 95.4% of the measured end-to-end latency at batch size 64 originated from CPU-side image loading and decoding. This finding shifted the optimization focus away from GPU computation and motivated a RAM-cached controlled experiment.

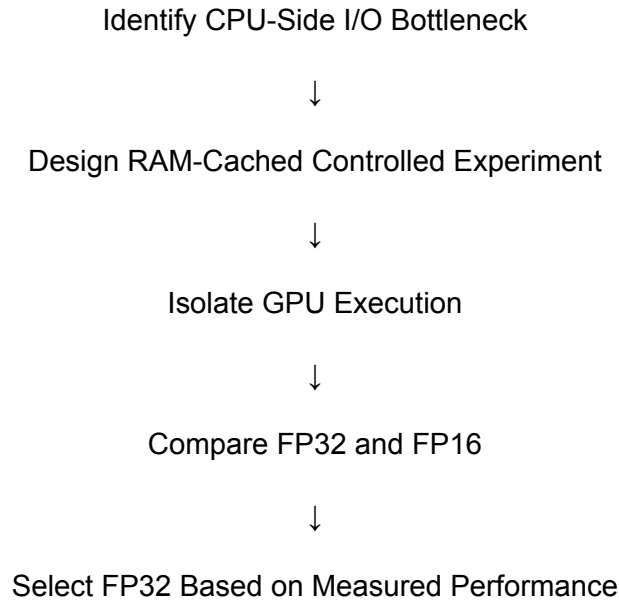
By removing repeated disk access from the timed execution path, the RAM-cached benchmark reduced measured end-to-end latency from 11.236 ms to 0.359 ms and provided a cleaner environment for evaluating GPU execution.

Under these controlled conditions, FP16 remained slower than FP32 across all tested batch sizes, while classification accuracy remained nearly unchanged. This result showed that reduced precision did not provide a practical advantage for the target workload.

Based on the measured evidence, FP32 was retained as the final inference configuration. The decision was therefore driven by observed workload behavior rather than assumptions based on theoretical GPU capability.

The optimization process can therefore be summarized as:





9. Limitations and Future Work

The results in this report are specific to the evaluated hardware, model architecture, runtime configuration, and workload. The target network is a lightweight CNN operating on small 24×24 wafer maps, and the relative performance of FP32 and FP16 may differ substantially for larger neural networks or more compute-intensive workloads.

In addition, the RAM-cached benchmark intentionally removes repeated disk access to isolate inference performance. While this is useful for controlled analysis, a production inference service would require additional consideration of data ingestion, request scheduling, concurrency, queueing, and memory management.

Future work could extend the system from static batch execution to a request-driven inference workload. In particular, dynamic or adaptive batching could be evaluated under bursty request arrival patterns to study the trade-off between throughput and request latency. Such an extension would move the project from isolated inference optimization toward the design of an inference serving system.

10. Conclusion

This work presented a profiling-driven optimization study of a C++/CUDA inference pipeline for semiconductor wafer defect classification. By decomposing the execution pipeline into individual stages, the analysis identified CPU-side disk loading and image decoding as the dominant end-to-end bottleneck.

A RAM-cached controlled experiment isolated this overhead and enabled a more focused comparison of FP32 and FP16 GPU inference. Despite FP16-capable GPU execution, FP16

did not improve throughput for the lightweight CNN, while classification quality remained nearly identical between the two precision modes.

The final decision to retain FP32 illustrates the central finding of this study: effective ML inference optimization requires measurement, bottleneck isolation, and workload-specific evaluation. Hardware capabilities can motivate optimization hypotheses, but system configuration should ultimately be selected based on empirical performance.